

TECHNISCHE DOKUMENTATION · SUPERMICRO X13SAE-F

Kühlung der Tesla T4 am X13SAE-F

Wie die Lüfterregelung für die Grafikkarte auf diesem Board gebaut wurde, welche Eigenheiten der Hardware dabei im Weg standen, und was man wissen muss, um dasselbe unter einem beliebigen Linux nachzubauen.

Stand 27. August 2026 **Alle Werte** auf diesem System gemessen

Übertragbar auf jedes Linux mit ipmitool

01 Das Problem

Die Tesla T4 ist eine Serverkarte ohne eigenen Lüfter. Im Rechenzentrum bläst das Gehäuse einen kräftigen Luftstrom längs durch ihren Kühlkörper. In einem Workstation-Gehäuse gibt es diesen Luftstrom nicht, deshalb sitzt hier ein Radiallüfter direkt an der Karte.

Der Haken: **Der Fernwartungsbaustein des Boards kennt die Temperatur der Grafikkarte nicht.** Er regelt nach Prozessor- und Systemsensoren. Die Karte kann glühen, ohne dass sich am Lüfter etwas tut – und umgekehrt dreht der Lüfter hoch, wenn der Prozessor arbeitet, obwohl die Karte kalt ist.

Die Lösung ist eine eigene Regelung, die die Temperatur der Karte ausliest und die Lüfter über den Fernwartungsbaustein stellt. Das ist der Kern dieses Dokuments. Drumherum liegen drei Eigenheiten der Hardware, ohne deren Behandlung die Regelung nichts nützt.

ERGEBNIS

Im Leerlauf läuft der Radiallüfter mit rund 3 200 von 6 650 möglichen Umdrehungen, die Karte liegt bei 38 bis 45 °C. Unter Dauerlast geht er auf Vollast und die Karte stabilisiert sich bei 72 °C – 13 Grad unter ihrer Betriebsgrenze. Über vier Tage Dauerbetrieb hat die Regelung genau einen Stellwert geändert.

02 Randbedingungen der Hardware

Was das Board vorgibt und woran sich jede Lösung halten muss.

LÜFTERANSCHLÜSSE UND IHRE ZUORDNUNG		
ZONE	ANSCHLÜSSE	WAS DARAN HÄNGT
Zone0	CPU_FAN1	Prozessorkühler
Zone0	CPU_FAN2	Radiallüfter der Grafikkarte
Zone1	SYS_FAN1-3	Gehäuselüfter

DIE ENTSCHEIDENDE EINSCHRÄNKUNG

Der Fernwartungsbaustein regelt nur zonenweise, nicht je Anschluss. Der Radiallüfter der Karte teilt sich Zone 0 also zwangsläufig mit dem Prozessorkühler. Beide bekommen immer denselben Stellwert. Alle fünf Anschlüsse des Boards sind belegt, ein freier ist nicht vorhanden.

Daraus folgt die wichtigste Entwurfsentscheidung: Zone 0 muss nach dem *höheren Bedarf* aus Grafikkarte und Prozessor geregelt werden. In der Praxis führt fast immer die Karte, weil sie deutlich früher warm wird. Der Prozessorkühler dreht dann mit, ohne es zu brauchen — das ist der Preis der Zonenbindung und in Kauf zu nehmen, weil der Prozessorkühler der leiseste der fünf ist.

Wer die Kopplung auflösen will, müsste den Prozessorkühler auf einen Gehäuse-Anschluss umstecken, damit Zone 0 allein der Karte gehört. Dafür wäre ein Y-Kabel nötig, um einen Anschluss frei zu bekommen, und die Ausfallschwelle von CPU_FAN1 müsste angepasst werden, weil ein leerer Anschluss als Lüfterausfall gilt. Hier wurde darauf verzichtet.

03 Das IPMI-Protokoll dieses Boards

Herstellereigene Befehle von Supermicro. Sie funktionieren unter jedem Linux mit `ipmitool`, unabhängig von der Distribution.

Lüftermodus lesen und setzen

```
ipmitool raw 0x30 0x45 0x00      # lesen, Antwort "01" = Full
ipmitool raw 0x30 0x45 0x01 0x01 # auf Full setzen
```

Ohne den Modus *Full* regelt der Baustein nach seiner eigenen Kurve weiter und überschreibt gesetzte Stellwerte binnen Sekunden. Zwei weitere Modi existieren (Standard, Optimal), beide sind hier unbrauchbar.

NICHT BLIND SETZEN

Der *Wechsel* nach Full lässt den Baustein zunächst alle Lüfter auf Vollast gehen. Steht der Modus ohnehin schon, erzeugt ein erneutes Setzen also einen unnötigen Vollast-Sprung. Erst lesen, dann nur bei Abweichung schreiben.

Stellwert setzen und zurücklesen

```
ipmitool raw 0x30 0x70 0x66 0x01 <zone> <wert>  # setzen
ipmitool raw 0x30 0x70 0x66 0x00 <zone>          # zuruecklesen

# Zone 0 auf 40 Prozent:
ipmitool raw 0x30 0x70 0x66 0x01 0x00 0x28
```

Der Wert ist ein Prozentsatz von `0x00` bis `0x64`, also 0 bis 100.

DREI EIGENHEITEN, DIE BEIM ZURÜCKLESEN VERWIRREN

BEOBACHTUNG	ERKLÄRUNG
Der zurückgelesene Wert liegt einen Punkt unter dem gesetzten . 55 kommt als <code>0x36</code> (54), 30 als <code>0x1d</code> (29).	Der Baustein rechnet intern auf 0–255 und wieder zurück. Rundung, kein Fehler. Beim Vergleichen also eine Toleranz von 1 einplanen.
Über <code>0x64</code> geht nichts. <code>0x80</code> , <code>0xc8</code> und <code>0xff</code> kommen alle als <code>0x64</code> zurück, die Drehzahl ändert sich nicht.	Harte Begrenzung. Es liegt keine verborgene Reserve über 100 % brach.
Jede gemessene Drehzahl ist ein Vielfaches von 140.	Auflösung des Drehzahlmessers. Erklärt auch, warum gesetzte Ausfallschwellen auf krumme Werte springen.

Drehzahlen und Temperaturen lesen

```
ipmitool sdr type Fan      # alle fuenf Anschluesse
ipmitool sdr type Temperature # Board-Sensoren
```

NICHT SCHNELL ABFRAGEN

Der Steuerkanal zwischen Betriebssystem und Fernwartungsbaustein (KCS) ist empfindlich. Mehrere `ipmitool` -Aufrufe binnen weniger Sekunden liefern gelegentlich Unsinn – Drehzahlen von 19 000, oder die Meldung `unexpected ID` . **Ein bis zwei Sekunden Pause zwischen den Aufrufen**, und Temperaturen besser gar nicht über IPMI lesen, sondern über die Kernel-Sensoren (siehe Abschnitt 6).

04 Drei Voraussetzungen

Ohne diese drei Einstellungen nützt die beste Regelung nichts. Alle drei sind unabhängig von der Distribution und müssen bei jedem Systemstart gesetzt werden.

Erstens: die Karte vor dem Ruhezustand bewahren

Nach dem Start meldet `lspci` die Karte, aber der Treiber findet sie nicht (`No such device`), und im Kernelprotokoll steht:

```
Unable to change power state from D3cold to D0, device inaccessible
NVRM: GPU has fallen off the bus
```

Die Energieverwaltung legt die Karte in den tiefsten Ruhezustand, und der PCIe-Anschluss am Chipsatz weckt sie nicht wieder. Im laufenden Betrieb ist nichts mehr zu retten — weder ein Neuladen des Treibers noch Entfernen und Neusuchen des Geräts. Erst der nächste Kaltstart bringt sie zurück.

```
echo 0 > /sys/bus/pci/devices/0000:08:00.0/d3cold_allowed
echo on > /sys/bus/pci/devices/0000:08:00.0/power/control
```

Muss **früh** laufen, solange die Karte noch wach ist. Die PCI-Adresse `0000:08:00.0` gilt für diese Bestückung; bei Umbauten mit `lspci -D | grep -i tesla` prüfen.

Nebenbefund: Von den beiden Zeilen trägt im laufenden Betrieb nur die zweite. Mit `power/control=on` ist die Laufzeit-Energieverwaltung abgeschaltet, das Gerät wird also ohnehin nie schlafen gelegt — eine halbe Stunde mit `d3cold_allowed=1` blieb folgenlos. Für das Zeitfenster beim Start bleibt die erste Zeile trotzdem nötig.

Zweitens: die Ausfallschwellen der Lüfter entschärfen

Ab Werk liegen die unteren Drehzahlschwellen so hoch, dass ein langsam drehender Lüfter als ausgefallen gilt. Der Baustein zwingt dann *alle* Zonen auf Vollast — und zwar dauerhaft, bis die Schwelle geändert wird. Der Prozessorkühler dreht im Leerlauf rund 560 Umdrehungen und lag damit werksseitig genau in diesem Bereich.

```
for f in CPU_FAN1 CPU_FAN2 SYS_FAN1 SYS_FAN2 SYS_FAN3; do
  ipmitool sensor thresh "$f" lower 0 100 200
done
```

Die drei Zahlen sind untere Schwellen für *non-recoverable*, *critical* und *non-critical*. Was danach tatsächlich im Sensor steht, ist nicht 0/100/200, sondern 0 / 140 / **nicht gesetzt** — der Baustein rastet auf sein 140er-Raster. Wirksam ist es trotzdem; man darf nur nicht nach den geschriebenen Zahlen suchen.

BLEIBT GESPEICHERT

Diese Schwellen überleben Neustarts im Fernwartungsbaustein. Ein Zurücksetzen auf Werkseinstellungen bringt das Problem aber zurück, deshalb gehört der Befehl trotzdem in den Systemstart.

Drittens: den Persistence Mode einschalten

```
nvidia-smi -pm 1
```

Ohne ihn baut der Grafiktreiber seinen Zustand ab, sobald kein Programm die Karte mehr geöffnet hält. Die Karte bleibt dann stehen, wo sie war – auf vollem Speichertakt. Sie findet **nicht von selbst** in einen Sparzustand zurück.

LEERLAUF MIT UND OHNE PERSISTENCE MODE, GEMESSEN				
ZUSTAND		SPEICHERTAKT	LEISTUNG	TEMPERATUR
ohne	P0	5 000 MHz	28,6 W	47–50 °C
mit	P8	405 MHz	9,6 W	38–42 °C

Das sind rund 19 Watt und 8 Grad Dauerunterschied im Leerlauf. Der Befehl muss **nach** dem Laden des Grafiktreibers laufen; beim Systemstart kommt der oft später als die Startskripte, deshalb lohnt eine Warteschleife.

WORAN MAN ES ERKENNT

nvidia-smi zeigt im gesunden Leerlauf P8 , 405 MHz und rund 10 Watt. Steht dort P0 mit 5 000 MHz und knapp 30 Watt, fehlt der Persistence Mode – auch wenn die Karte sonst normal arbeitet.

05 Die Messwerte

Damit niemand nachmessen muss. Alles auf diesem System erhoben, bei rund 25 °C Raumtemperatur.

Stellwert gegen Drehzahl

BEIDE ZONEN GEMEINSAM, JE 20 S EINSCHWINGZEIT, ANGABEN IN U/MIN					
STELLWERT	RADIALLÜFTER	CPU-KÜHLER	GEHÄUSE 1	GEHÄUSE 2	GEHÄUSE 3
100 %	6 580	3 080	1 120	1 680	1 400
90 %	6 440	2 940	980	1 680	1 400
80 %	6 020	2 660	840	1 540	1 260
70 %	5 460	2 240	840	1 400	1 120
60 %	5 180	1 960	700	1 260	980
50 %	4 620	1 680	560	1 120	840
40 %	3 920	1 260	420	980	700
30 %	3 220	840	420	700	560
25 %	2 800	700	280	560	420
20 %	Lüfterausfall gemeldet, alle Zonen auf Vollast				

UNTERGRENZE

Bei 20 % fällt SYS_FAN1 unter die Ausfallschwelle von 140 Umdrehungen. Zone 1 darf deshalb nie unter 25 %. In der Regelung sind 30 % als Untergrenze gesetzt, mit Sicherheitsabstand. Zone 0 hätte technisch mehr Luft nach unten, läuft aber aus Gleichlauf ebenfalls nicht unter 30 %.

Verhalten unter Vollast

DAUERLAST AUF DER KARTE BEI 68 W, BEHARRUNGSTEMPERATUR JE STELLWERT			
STELLWERT ZONE 0	RADIALLÜFTER	TEMPERATUR	BEWERTUNG
100 %	6 580	72 °C	tragfähig
70 %	5 460	79 °C	grenzwertig
50 %	4 620	84 °C	Messung abgebrochen

LEERLAUF 38–45 °C	VOLLLAST 72 °C	BETRIEBSGRENZE 85 °C	DROSSELUNG AB 93 °C
ZEITKONSTANTE 5–6 min	MAX. DREHZAHL 6 650 U/min		

Zwei Folgerungen für den Entwurf. Erstens: **Die Kurve muss spätestens bei 72 °C auf 100 % stehen**, denn das ist der Beharrungspunkt bei maximaler Kühlung. Darüber gibt es nichts mehr zu regeln. Zweitens: Die thermische Zeitkonstante von fünf bis sechs Minuten macht einen schnellen Regeltakt entbehrlich – zehn Sekunden reichen mit großem Abstand.

Der Takt darf trotzdem nicht auf eine Minute gehen. Beim Lastsprung von 10 auf 68 Watt steigt die Temperatur anfangs mit mehreren Grad je Minute, und ein Minutentakt gäbe dem entsprechend Vorlauf.

06 Entwurf der Regelung

Fünf Entscheidungen und ihre Begründung.

Getrennte Kurven, Maximum der Stellwerte

Zone 0 wird nach `max(kurve_gpu(gpu), kurve_cpu(cpu))` geregelt — nicht nach `kurve(max(gpu, cpu))`. Der Unterschied ist wesentlich: 43 °C am Prozessor und 72 °C an der Karte sind keine vergleichbaren Zahlen. Für den Prozessor sind 43 Grad kalt, für die Karte sind 72 nahe an der Grenze. Erst die Umrechnung in einen Stellwert macht beide vergleichbar.

Hysterese am Stellwert, nicht an der Temperatur

DER FEHLER, DER HIER ZWEIMAL GEMACHT WURDE

Eine frühere Fassung merkte sich beim Hochschalten die aktuelle Temperatur und schaltete erst herunter, wenn sie 3 Grad darunter lag. Klingt richtig, war es aber nicht: Die gespeicherte Referenz wurde bei *jedem* Durchlauf überschrieben und wanderte deshalb mit der Temperatur mit. Heruntergeschaltet wurde nur noch, wenn die Temperatur binnen eines Durchlaufs um mehr als 3 Grad fiel — bei normaler Abkühlung also nie. Der Lüfter blieb oben stehen.

Die Lösung hängt die Hysterese an den *Stellwert*: Nach oben wird sofort gefolgt, nach unten erst ab 4 Punkten Differenz und dann höchstens 5 Punkte je Schritt. Damit gibt es keine wandernde Referenz mehr.

Die Zahlen hängen an der Skala. Auf einem Vergleichssystem mit einer Skala von 0 bis 255 sind 8 Punkte Hysterese passend; auf der Skala 0 bis 100 dieses Boards sind dieselben 8 Punkte fast dreimal so grob und lassen den Lüfter merklich über dem Sollwert stehen.

Glättung, die nur nach unten wirkt

Über die letzten sechs Messungen läuft ein gleitender Mittelwert. Für den Stellwert wird aber das *Maximum* aus geglättetem und rohem Bedarf genommen. Ein einzelner Ausreißer nach oben hebt die Stufe also sofort, senkt sie aber nicht — Trägheit dort, wo sie hilft, keine Trägheit dort, wo sie schadet.

Lesen ohne IPMI

Prozessor- und Datenträgertemperaturen kommen aus den Kernel-Sensoren (`coretemp` und `drivetemp` unter `/sys/class/hwmon`), die Karte über `nvidia-smi`. IPMI wird nur zum Schreiben benutzt, und nur bei Änderung oder alle 30 Sekunden zur Auffrischung. Das umgeht die Empfindlichkeit des Steuerkanals gegen schnelles Abfragen vollständig.

Für Zone 1 sind die Datenträgertemperaturen die ehrlichere Führungsgröße als der Board-Sensor „System Temp“: Die Gehäuselüfter kühlen den Laufwerkskäfig. Der Board-Sensor zeigte 34 °C, während der wärmste Datenträger bei 40 lag.

Alles Unbekannte führt zu Vollast

Antwortet `nvidia-smi` nicht, geht Zone 0 auf 100 %. Fehlt ein Sensorwert, fällt er aus der Betrachtung; fehlen alle, gilt ebenfalls Vollast. Dazu feste Grenzen: Karte ab 80 °C, Prozessor ab 92 °C, Datenträger ab 58 °C führen sofort auf Vollast, unabhängig von der Kurve. Beim Beenden setzt der Dienst beide Zonen auf Vollast, bevor er geht.

EIN WÄCHTER IST HIER PFLICHT, NICHT KOMFORT

Sobald der Lüftermodus auf *Full* steht und die Ausfallschwellen entschärft sind, hält **keine Hardware mehr von unten dagegen**. Stirbt oder hängt die Regelung, friert der zuletzt geschriebene Stellwert ein — auch wenn er 30 % beträgt und die Karte danach unter Last geht. Übrig bliebe nur die Drosselung der Karte bei 93 °C. Ein Wächter, der Prozess *und* Lebenszeichen prüft, gehört zwingend dazu.

07 Referenzfassung

Lauffähige Kurzfassung. Braucht `ipmitool`, `nvidia-smi` und die Kernelmodule `coretemp` und `drivetemp`, sonst nichts. Läuft als `root`.

```
#!/usr/bin/env python3
"""Luefterregelung fuer Supermicro X13SAE-F mit NVIDIA Tesla T4.

Portable Kurzfassung: laeuft auf jedem Linux mit ipmitool, nvidia-smi und den
Kernelmodulen coretemp und drivetemp. Voraussetzung sind die drei Einstellungen
aus dem Abschnitt "Drei Voraussetzungen" - ohne die nuetzt die Regelung nichts.
"""
import os
import subprocess
import signal
import sys
import time
from collections import deque

INTERVAL    = 10    # Sekunden je Regelschritt
SMOOTH_N    = 6     # gleitender Mittelwert ueber 6 Messungen
HYST        = 4     # erst ab 4 Punkten Differenz herunterschalten
DOWN_STEP   = 5     # und dann hoechstens 5 Punkte je Schritt
REASSERT    = 30    # Stellwert spaetestens alle 30 s erneut schreiben
Z0_MIN      = 30    # Untergrenze Zone 0
Z1_MIN      = 30    # Untergrenze Zone 1 - unter 25 % droht Fan-Fault
MAXD        = 100

CURVE_GPU   = [(45, 30), (55, 50), (62, 70), (68, 90), (72, 100)]
CURVE_CPU   = [(55, 30), (70, 45), (80, 65), (88, 85), (95, 100)]
CURVE_DISK  = [(40, 30), (45, 50), (50, 70), (55, 90), (58, 100)]
EMERG       = {"gpu": 80, "cpu": 92, "disk": 58}

HEARTBEAT   = "/tmp/t4-fan.heartbeat"

def ipmi(*args):
    try:
        r = subprocess.run(["ipmitool"] + list(args),
                           capture_output=True, text=True, timeout=15)
        return r.returncode == 0, r.stdout
    except Exception:
        return False, ""

def set_duty(zone, duty):
    """Bis zu drei Versuche mit Pause - der KCS-Kanal ist empfindlich."""
    for _ in range(3):
        ok, _ = ipmi("raw", "0x30", "0x70", "0x66", "0x01",
                     "0x%02x" % zone, "0x%02x" % duty)
        if ok:
            return True
        time.sleep(2)
    return False

def fan_mode_full():
    """Nur setzen, wenn noetig: Der Wechsel nach Full laesst den BMC
    zunaechst alle Luefter auf Vollast gehen."""
    ok, out = ipmi("raw", "0x30", "0x45", "0x00")
    if ok and out.replace(" ", "").strip() == "01":
```

```

        return
    ipmi("raw", "0x30", "0x45", "0x01", "0x01")
    time.sleep(3)

def hwmon_max(name):
    """Höchste Temperatur aller Sensoren eines hwmon-Treibers."""
    best = -1
    basis = "/sys/class/hwmon"
    try:
        eintraege = os.listdir(basis)
    except OSError:
        return best
    for d in eintraege:
        pfad = os.path.join(basis, d)
        try:
            with open(os.path.join(pfad, "name")) as fh:
                if fh.read().strip() != name:
                    continue
            for f in os.listdir(pfad):
                if f.startswith("temp") and f.endswith("_input"):
                    with open(os.path.join(pfad, f)) as fh:
                        v = int(fh.read().strip()) // 1000
                        if 0 < v < 150:
                            best = max(best, v)
        except (OSError, ValueError):
            continue
    return best

def gpu_temp():
    try:
        r = subprocess.run(["nvidia-smi", "--query-gpu=temperature.gpu",
                            "--format=csv,noheader,nounits"],
                            capture_output=True, text=True, timeout=10)
        werte = [int(x) for x in r.stdout.split() if x.strip().isdigit()]
        return max(werte) if werte else -1
    except Exception:
        return -1

def curve(punkte, temp):
    """Lineare Interpolation zwischen den Stuetzpunkten."""
    if temp is None or temp < 0:
        return None
    if temp <= punkte[0][0]:
        return punkte[0][1]
    if temp >= punkte[-1][0]:
        return punkte[-1][1]
    for (t0, d0), (t1, d1) in zip(punkte, punkte[1:]):
        if t0 <= temp <= t1:
            return int(d0 + (d1 - d0) * (temp - t0) / (t1 - t0))
    return punkte[-1][1]

def next_duty(aktuell, ziel, minimum):
    """Nach oben sofort, nach unten gebremst.

    Die Hysteresse haengt am Stellwert, nicht an einer gespeicherten
    Temperatur. Wer sie an die letzte Messung bindet, bekommt eine Referenz,
    die mit der Temperatur mitwandert - dann wird nie heruntergeschaltet.
    """
    if aktuell is None:
        return ziel

```

```

    if ziel > aktuell:
        return ziel
    if aktuell - ziel < HYST:
        return aktuell
    return max(ziel, aktuell - DOWN_STEP, minimum)

def vollast(signum=None, frame=None):
    fan_mode_full()
    set_duty(0, MAXD)
    time.sleep(1)
    set_duty(1, MAXD)
    sys.exit(0)

signal.signal(signal.SIGTERM, vollast)
signal.signal(signal.SIGINT, vollast)

def main():
    fan_mode_full()
    puffer = {k: deque(maxlen=SMOOTH_N) for k in ("gpu", "cpu", "disk")}
    duty0 = duty1 = None
    letzte_schreibung = 0.0

    while True:
        roh = {"gpu": gpu_temp(),
              "cpu": hwmon_max("coretemp"),
              "disk": hwmon_max("drivetemp")}
        for k, v in roh.items():
            if v >= 0:
                puffer[k].append(v)
        mittel = {k: (sum(b) / len(b) if b else None) for k, b in puffer.items()}

        def bedarf(punkte, k):
            """Nach oben auf den Rohwert reagieren, nach unten auf den
            geglätteten: ein Ausreisser hebt die Stufe, senkt sie aber nicht."""
            werte = [x for x in (curve(punkte, mittel[k]), curve(punkte, roh[k]))
                     if x is not None]
            return max(werte) if werte else None

        # Zone 0: Maximum der Stellwerte, nicht der Temperaturen
        if roh["gpu"] < 0:
            ziel0 = MAXD          # Karte antwortet nicht - nie leise bei Unklarheit
        else:
            k0 = [x for x in (bedarf(CURVE_GPU, "gpu"), bedarf(CURVE_CPU, "cpu"))
                  if x is not None]
            ziel0 = max(k0) if k0 else MAXD

        k1 = [x for x in (bedarf(CURVE_DISK, "disk"), bedarf(CURVE_CPU, "cpu"))
              if x is not None]
        ziel1 = max(k1) if k1 else MAXD

        if any(roh[k] >= grenze for k, grenze in EMERG.items()):
            neu0 = neu1 = MAXD
        else:
            neu0 = next_duty(duty0, max(Z0_MIN, min(MAXD, ziel0)), Z0_MIN)
            neu1 = next_duty(duty1, max(Z1_MIN, min(MAXD, ziel1)), Z1_MIN)

        jetzt = time.time()
        faellig = (jetzt - letzte_schreibung) >= REASSERT
        if neu0 != duty0 or faellig:
            set_duty(0, neu0)
            time.sleep(1)

```

```
        if neu1 != duty1 or faellig:
            set_duty(1, neu1)
        if faellig:
            letzte_schreibung = jetzt
        duty0, duty1 = neu0, neu1

    try:
        with open(HEARTBEAT, "w") as fh:
            fh.write("%d %d %d\n" % (int(jetzt), neu0, neu1))
    except OSError:
        pass

    time.sleep(INTERVAL)

if __name__ == "__main__":
    try:
        main()
    except Exception:
        vollast()
```

Auf dem übergebenen System läuft eine etwas ausführlichere Variante desselben Aufbaus mit Protokollierung und Rotation unter `/mnt/scripts/t4-fan.py` .

08 Beim Systemstart ausführen

Vier Dinge müssen bei jedem Start passieren, in dieser Reihenfolge. Wie sie angestoßen werden, hängt vom System ab.

1. Ruhezustand der Karte sperren (früh, solange sie wach ist)
2. Lüftermodus auf *Full*, beide Zonen vorsichtshalber hoch setzen
3. Ausfallschwellen entschärfen
4. Persistence Mode setzen, sobald der Grafiktreiber bereit ist, und die Regelung starten

Auf einem gewöhnlichen Linux mit systemd

Eine Einheit für die Voraussetzungen:

```
[Unit]
Description=Voraussetzungen fuer Tesla T4 und Luefterregelung
After=multi-user.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/bin/sh -c 'echo 0 > /sys/bus/pci/devices/0000:08:00.0/d3cold_allowed'
ExecStart=/bin/sh -c 'echo on > /sys/bus/pci/devices/0000:08:00.0/power/control'
ExecStart=/usr/bin/ipmitool raw 0x30 0x45 0x01 0x01
ExecStart=/usr/bin/ipmitool raw 0x30 0x70 0x66 0x01 0x00 0x64
ExecStart=/usr/bin/ipmitool raw 0x30 0x70 0x66 0x01 0x01 0x64
ExecStart=/bin/sh -c 'for f in CPU_FAN1 CPU_FAN2 SYS_FAN1 SYS_FAN2 SYS_FAN3; do ipmitool
sensor thresh "$f" lower 0 100 200; sleep 1; done'
ExecStart=/bin/sh -c 'for i in $(seq 1 30); do nvidia-smi -pm 1 && exit 0; sleep 2; done;
exit 1'

[Install]
WantedBy=multi-user.target
```

Und eine für die Regelung selbst:

```
[Unit]
Description=Luefterregelung Tesla T4
Requires=t4-voraussetzungen.service
After=t4-voraussetzungen.service

[Service]
Type=simple
ExecStart=/usr/local/sbin/t4-fan.py
Restart=always
RestartSec=5
ExecStopPost=/usr/bin/ipmitool raw 0x30 0x70 0x66 0x01 0x00 0x64
ExecStopPost=/usr/bin/ipmitool raw 0x30 0x70 0x66 0x01 0x01 0x64

[Install]
WantedBy=multi-user.target
```

`Restart=always` fängt den abgestürzten Prozess ab, `ExecStopPost` setzt dabei zuerst Vollast. Was `systemd` *nicht* abfängt, ist ein hängender Prozess — dafür prüft ein Zeitgeber das Lebenszeichen:

```
# t4-fan-waechter.service (Type=oneshot)
ExecStart=/bin/sh -c 'test -f /tmp/t4-fan.heartbeat && \
  [ $(( $(date +%s) - $(stat -c %Y /tmp/t4-fan.heartbeat) )) -le 90 ] || \
  systemctl restart t4-fan.service'

# t4-fan-waechter.timer
[Timer]
OnBootSec=2min
OnUnitActiveSec=1min
```

Auf TrueNAS SCALE, wie übergeben

Dort gibt es kein direktes Bearbeiten von `systemd`-Einheiten, weil sie beim Update überschrieben würden. Stattdessen tragen die Weboberfläche unter *System* → *Erweitert* acht sogenannte `PostInit`-Einträge und ein `Cron`-Auftrag dieselbe Arbeit. Die Skripte liegen in einem eigenen Datenbereich, damit sie Updates überstehen.

NUR TRUENAS, ABER ÜBEL

TrueNAS reicht den hinterlegten Befehl durch eine zusätzliche Kommandozeilen-Ebene. **Alles mit einem Dollarzeichen wird schon vorher aufgelöst** — Schleifen über Variablen und Befehlsersetzungen scheitern, und zwar lautlos: weder im Startprotokoll noch in der Weboberfläche steht etwas. Nur `/var/log/middlewared.log` zeigt es, auffindbar über `grep InitShutdownScriptService`. Auf diesem System hatte ein Eintrag deshalb *nie* funktioniert, ohne dass es jemandem auffiel. Befehle dort also ohne Variablen schreiben, notfalls ausgeschrieben statt als Schleife.

09 Fallstricke beim Nachbauen

STOLPERSTEIN	WAS PASSIERT UND WAS HILFT
Lüftermodus blind setzen	Der Wechsel nach <i>Full</i> jagt zunächst alle Lüfter auf Vollast. Beim Start des Dienstes bleibt das stehen, bis die nächste Auffrischung greift. Erst lesen, nur bei Abweichung schreiben.
Zu schnelles Abfragen über IPMI	Liefert Müllwerte oder <code>unexpected ID</code> . Temperaturen über die Kernel-Sensoren lesen, IPMI nur zum Schreiben, Pausen zwischen Aufrufen.
Prozesssuche mit <code>pkill -f</code>	Das Muster steht auch in der eigenen Kommandozeile — man beendet sich selbst. Mit dem Klammertrick prüfen: <code>ps -eo cmd grep "[t]4-fan.py"</code> .
Skripte unter <code>/tmp</code> ablegen	Ist auf TrueNAS mit <code>noexec</code> eingehängt, Start scheitert auch als <code>root</code> (systemd meldet 203/EXEC). Gilt auf vielen gehärteten Systemen.
Der Zurücklesewert weicht ab	Immer einen Punkt niedriger als gesetzt. Wer exakt vergleicht, findet dauernd Abweichungen, die keine sind.
Aus Durchgangswerten Temperaturen ableiten	Beim schnellen Durchfahren der Stellwerte scheint die Karte kühler, als sie ist. Bei 25 % zeigte ein 20-Sekunden-Schritt 51 °C, der echte Beharrungswert lag bei 63.
Annehmen, eine Einstellung wirke	Um zu prüfen, ob ein Startbefehl wirklich etwas tut, die Einstellung vorher absichtlich verstellen. Sonst kann ein Befehl jahrelang als funktionierend gelten, obwohl er nur einen bereits richtigen Zustand vorfindet.

10 Prüfen ob es läuft

Ist die Karte gesund?

```
nvidia-smi --query-gpu=persistence_mode,pstate,clocks.mem,power.draw,temperature.gpu \
--format=csv,noheader
```

```
# Gesund im Leerlauf:      Enabled, P8, 405 MHz, ~10 W, 38-45 C
# Persistence Mode fehlt: Disabled, P0, 5000 MHz, ~29 W
```

Regelt die Steuerung?

```
ps -eo cmd | grep "[t]4-fan.py"           # Prozess da?
stat -c %y /tmp/t4-fan.heartbeat          # Lebenszeichen frisch?
ipmitool sdr type Fan                     # Drehzahlen
sleep 2; ipmitool raw 0x30 0x70 0x66 0x00 0x00 # Stellwert Zone 0
```

Der aussagekräftigste Test

Last auf die Karte geben und zusehen. Ohne installiertes Werkzeug genügt ein Container mit einem CUDA-Beispiel; das hier verwendete Abbild ist 125 MB groß:

```
docker run --rm --runtime=nvidia --network none \
-e NVIDIA_VISIBLE_DEVICES=all -e NVIDIA_DRIVER_CAPABILITIES=compute,utility \
nvcr.io/nvidia/k8s/cuda-sample:nbody \
nbody -benchmark -numbodies=131072 -i=2000
```

Erwartetes Bild: Die Leistung geht auf 66 bis 69 Watt, die Temperatur steigt, und der Stellwert zieht in Zehn-Sekunden-Schritten der Kurve folgend nach — im Test von 30 über 42, 55, 67, 80, 92 auf 100 %. Nach dem Abschalten fällt er in Fünferschritten wieder zurück. **Bleibt er oben stehen, ist die Hysterese falsch gebaut.**

Kühlung der Tesla T4 am Supermicro X13SAE-F · Stand 27. August 2026

Alle Zahlen wurden am 23., 24. und 27. August 2026 auf diesem System gemessen, nicht aus Datenblättern übernommen. Wo etwas ungeklärt blieb – etwa warum derselbe Radiallüfter an diesem Board bei gleichem Stellwert rund ein Viertel langsamer dreht als an einem Vergleichssystem – steht es als ungeklärt.

Berichtigung und Nachprüfung

Stand 18. September 2026 · ergänzt zur Ausgabe vom 27. August 2026

Pfad der Lebenszeichendatei

In der Ausgabe vom 27. August war die Lebenszeichendatei der Regelung mit `/run/t4-fan.heartbeat` angegeben: in der Referenzfassung (Abschnitt 7), im Wächter (Abschnitt 8) und im Prüfbefehl (Abschnitt 10). Auf dem System liegt die Datei jedoch unter `/tmp/t4-fan.heartbeat`.

Eingerichtet ist nichts fehlerhaft: Regelung und Wächter verwenden dort übereinstimmend denselben Pfad. Wer aber den Prüfbefehl aus Abschnitt 10 abtippt, erhält "No such file or directory" und hält den Wächter für ausgefallen. In dieser Ausgabe sind alle vier Fundstellen angeglichen.

Das `noexec` auf `/tmp` (Abschnitt 9) steht dem nicht entgegen, denn es verhindert das Ausführen von Dateien, nicht das Schreiben. Wer die Regelung auf einem anderen System nachbaut, kann ebenso `/run` nehmen, dann aber an beiden Stellen: in der Regelung und im Wächter.

Nachprüfung unter Last, 18. September 2026

Vor der Übergabe wurde die Regelung noch einmal vollständig durchgefahen. Die Last lag bei 69 Watt auf der Karte.

ZUSTAND	GPU	STELLWERT ZONE 0	RADIALLÜFTER
Leerlauf vor der Last	42 °C	30 %	3 220 U/min
Anstieg	59 °C	60 %	5 040 U/min
Anstieg	67 °C	85 %	6 160 U/min
Anstieg	71 °C	94 %	6 580 U/min
Beharrung unter Vollast	74 bis 75 °C	100 %	6 860 U/min
Rücklauf	55 °C	89 %	6 440 U/min
Rücklauf	47 °C	69 %	5 460 U/min
Rücklauf	43 °C	44 %	4 200 U/min
Leerlauf nach der Last	42 °C	30 %	3 220 U/min

Der Stellwert folgte der Kennlinie aus Abschnitt 6 in beide Richtungen und fiel nach dem Abschalten in Fünferschritten bis auf die Untergrenze zurück. Die Hysterese arbeitet also wie beschrieben. Der Persistence Mode hielt auch nach Vollast: die Karte fiel sofort wieder auf P8 bei 9,6 Watt.

Zwei Abweichungen gegenüber den Augustwerten, beide ohne Folgen für den Betrieb. Die Karte lag unter Vollast bei 74 bis 75 °C statt 72 °C, bei weiterhin zehn Grad Abstand zur Betriebsgrenze von 85 °C und achtzehn Grad bis zur Drosselung. Und der Radiallüfter drehte bei Vollast 6 860 statt 6 580 U/min, also zwei Stufen des 140er-Rasters höher.